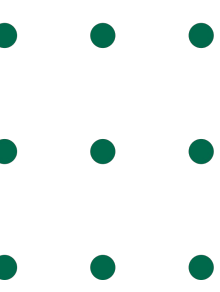
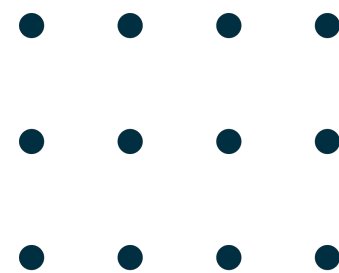


O que é são Hiperparâmetros?

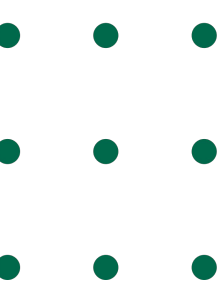
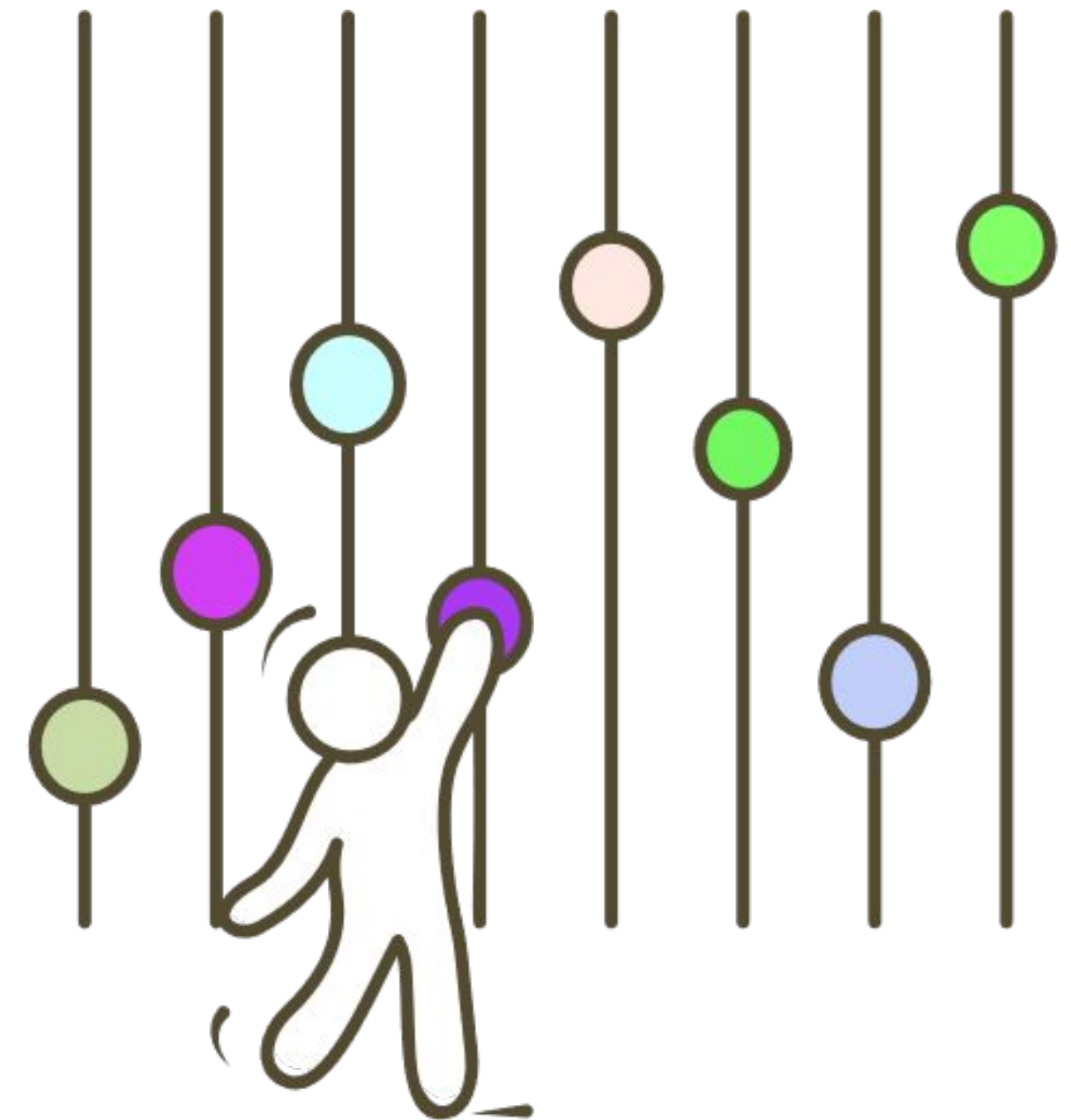
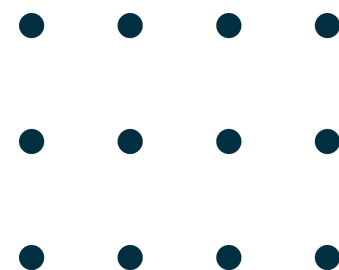
Hiperparâmetros são configurações externas a um modelo de aprendizado de máquina, definidas antes do processo de treinamento. Eles controlam o comportamento do algoritmo e têm um grande impacto no desempenho final do modelo.

Os hiperparâmetros influenciam diretamente a capacidade do modelo de aprender e generalizar a partir dos dados. Ajustar os hiperparâmetros corretamente pode significar a diferença entre um modelo que tem um bom desempenho e um que não consegue fazer boas previsões.



Hiperparâmetros X Parâmetros

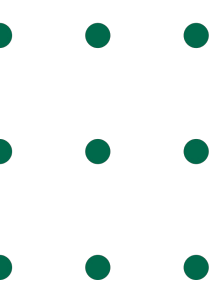
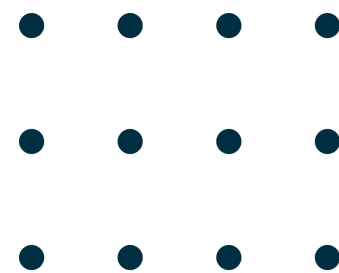
- **Parâmetros:** Valores aprendidos pelo modelo durante o treinamento. Por exemplo, os coeficientes em uma regressão linear ou os pesos em uma rede neural.
- **Hiperparâmetros:** Valores definidos antes do treinamento que não são aprendidos pelo modelo. Exemplos incluem a taxa de aprendizado (learning rate) em redes neurais, o número de árvores em uma floresta aleatória (Random Forest), ou o número de clusters em K-means.



Quando Utilizar?

Quando:

- **Durante o Desenvolvimento do Modelo:** Antes de iniciar o treinamento do modelo, é necessário definir os hiperparâmetros. Ajustar esses valores é essencial para garantir que o modelo tenha um bom desempenho nos dados de validação.
- **Otimização do Modelo:** Ao perceber que o modelo não está performando bem, ajustar os hiperparâmetros pode ajudar a melhorar a precisão, reduzir o overfitting ou underfitting.





Regressão Logística

C, SOLVER E PENALTY

C (Regularização)

- Descrição: O hiperparâmetro C controla a força da regularização. Regularização é uma técnica usada para evitar overfitting, penalizando grandes coeficientes no modelo.
- Interpretação: Um valor pequeno de C implica uma regularização mais forte, enquanto um valor grande de C implica uma regularização mais fraca.
- A regularização adiciona um termo de penalização à função de custo do modelo, incentivando a simplicidade e evitando que o modelo se ajuste excessivamente aos dados de treinamento.
- 2 principais, Lasso e Ridge.

model = LogisticRegression(C=1.0)

solver (Algoritmo de Otimização)

- Descrição: O hiperparâmetro solver especifica o algoritmo a ser usado na otimização do problema de regressão logística.
- Valores Comuns:
 - 'liblinear': Um resolvedor bom para pequenos conjuntos de dados.
 - 'lbfgs': Um resolvedor robusto que funciona bem na maioria dos casos.
 - 'newton-cg': Outro resolvedor robusto que pode ser usado para grandes conjuntos de dados.
 - 'sag': Um resolvedor otimizado para grandes conjuntos de dados.

model = LogisticRegression(solver='lbfgs')



Regressão Logística

C, SOLVER E PENALTY

penalty (Penalização)

- Descrição: O hiperparâmetro penalty especifica a norma usada na penalização (regularização) do modelo.
- Valores Comuns:
 - 'l2': Penalização de norma L2 (também conhecida como ridge).
 - 'l1': Penalização de norma L1 (também conhecida como lasso), que pode ser usada para realizar seleção de características.
 - 'elasticnet': Combinação de L1 e L2.
 - 'none': Sem penalização.

model = LogisticRegression(penalty='l2')



Árvores de

Max, depth, min_samples_split,
Decisão
min_samples_leaf

max_depth (Profundidade Máxima)

- Descrição: Define a profundidade máxima da árvore, ou seja, o número máximo de níveis que a árvore pode ter.
- Impacto: Controla o overfitting e underfitting. Uma árvore muito profunda pode levar a overfitting (captura muito bem os dados de treinamento, mas não generaliza bem). Uma árvore muito rasa pode levar a underfitting (não captura bem a estrutura dos dados). Impacto: Controla o overfitting e underfitting. Uma árvore muito profunda pode levar a overfitting (captura muito bem os dados de treinamento, mas não generaliza bem). Uma árvore muito rasa pode levar a underfitting (não captura bem a estrutura dos dados).

```
model = DecisionTreeClassifier(max_depth=5)
```

min_samples_split (Número Mínimo de Amostras para Dividir um Nó)

- Descrição: Define o número mínimo de amostras necessárias para dividir um nó interno. Se um nó possui menos amostras do que esse valor, ele não será dividido.
- Impacto: Controla o crescimento da árvore. Valores altos evitam que a árvore se divida com dados pequenos e ruidosos, ajudando a evitar overfitting.

model =

```
DecisionTreeClassifier(min_samples_split=10)
```

02

Árvores de

Max, depth, min_samples_split,
Decisão
min_samples_leaf

min_samples_leaf (Número Mínimo de Amostras em uma Folha)

- Descrição: Define o número mínimo de amostras que cada nó folha deve ter. Garante que cada folha tenha pelo menos esse número de amostras.
- Impacto: Controla o tamanho das folhas. Valores altos podem suavizar o modelo, fazendo com que ele generalize melhor e reduza o overfitting.

model = DecisionTreeClassifier(min_samples_leaf=5)



K-means

n_clusters e max_iter

n_clusters (Número de Clusters)

- Descrição: Define o número de clusters que o algoritmo deve formar.
- Impacto: Este é o hiperparâmetro mais importante, pois determina a estrutura do agrupamento. Um valor muito baixo pode resultar em clusters grandes e heterogêneos, enquanto um valor muito alto pode resultar em clusters pequenos e fragmentados. Este é o hiperparâmetro mais importante, pois determina a estrutura do agrupamento. Um valor muito baixo pode resultar em clusters grandes e heterogêneos, enquanto um valor muito alto pode resultar em clusters pequenos e fragmentados.

max_iter (Número Máximo de Iterações)

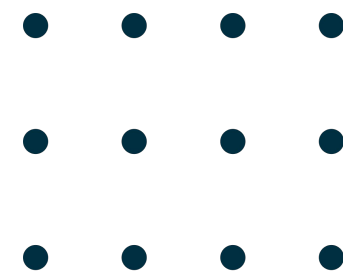
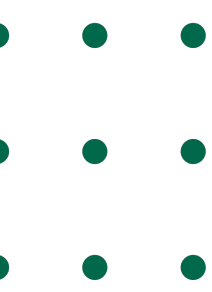
- Descrição: Define o número máximo de iterações do algoritmo K-Means para uma única execução. Define o número máximo de iterações do algoritmo K-Means para uma única execução.
- Impacto: Se o algoritmo não convergir antes desse número de iterações, ele será interrompido. Um valor muito baixo pode impedir a convergência, enquanto um valor muito alto pode resultar em tempo de execução mais longo sem benefícios adicionais.

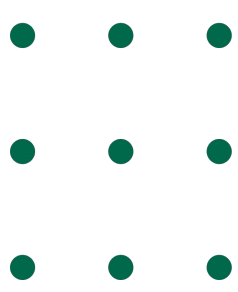
```
model = KMeans(n_clusters=3, max_iter=300)
```


Grid Search e Random Search

Grid Search e Random Search são técnicas usadas para otimização de hiperparâmetros em modelos de aprendizado de máquina. Ambas as técnicas buscam encontrar a combinação ideal de hiperparâmetros que maximiza o desempenho do modelo, mas o fazem de maneiras diferentes. Grid Search é uma técnica exaustiva que experimenta todas as combinações possíveis de hiperparâmetros em uma grade definida pelo usuário.

Random Search experimenta uma seleção aleatória de combinações de hiperparâmetros dentro de um espaço de busca definido pelo usuário. Random Search experimenta uma seleção aleatória de combinações de hiperparâmetros dentro de um espaço de busca definido pelo usuário.

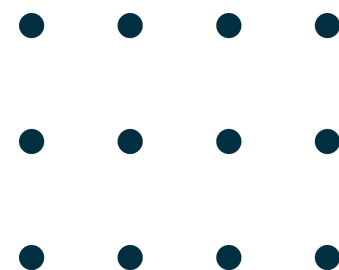


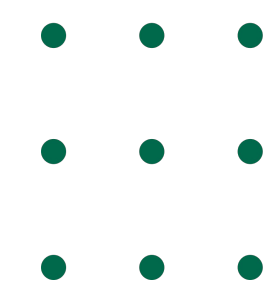


Grid Search

```
grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=5, scoring='accuracy')
```

- **GridSearchCV:** Realiza uma busca exaustiva sobre um espaço de hiperparâmetros especificado para encontrar a melhor combinação.
 - **estimator:** O modelo a ser ajustado.
 - **param_grid:** O espaço de hiperparâmetros a ser pesquisado.
 - **cv:** O número de folds para validação cruzada.
 - **scoring:** A métrica usada para avaliar o desempenho do modelo.
- 





Random Search

```
random_search = RandomizedSearchCV(estimator=model, param_distributions=param_dist, n_iter=100,  
                                   cv=5, scoring='accuracy', random_state=42)
```

- **RandomizedSearchCV:** Realiza uma busca aleatória sobre um espaço de hiperparâmetros especificado para encontrar a melhor combinação.
 - **estimator:** O modelo a ser ajustado.
 - **param_distributions:** O espaço de hiperparâmetros a ser explorado, com distribuições ou listas de valores possíveis.
 - **n_iter:** O número de combinações aleatórias de hiperparâmetros a serem testadas.
 - **cv:** O número de folds para validação cruzada.
 - **scoring:** A métrica usada para avaliar o desempenho do modelo.
 - **random_state:** Garante a reprodutibilidade dos resultados, fixando a semente do gerador de números aleatórios.
- 
- 